

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

3 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Borys Alexy Panezo Guerrero

PARALELO: A

SEMESTRE: Quinto

ACTIVIDAD: # 9 — Proyecto Práctico: Frontend e Integración Full Stack

PERIODO ABRIL - AGOSTO 2026



Actividad # 9: Desarrollo del Frontend e Integración Full Stack

Objetivo

Construir una Single Page Application (SPA) interactiva con Angular, aplicando modularización y consumiendo la API REST desarrollada en la Actividad 8, para completar un sistema web Full Stack desplegado en producción.

Tecnologías utilizadas

- Angular 22 (standalone components, signals, control flow `@if/@for`) para el frontend.
- TypeScript, RxJS y Reactive Forms para la lógica de la interfaz.
- HttpClient con un interceptor de autenticación HTTP Basic.
- Java 21 / Spring Boot / PostgreSQL / Flyway para el backend (Actividad 8).
- Docker para empaquetar el backend en producción.
- Render para desplegar el backend y la base de datos.
- Vercel para publicar el frontend como sitio estático.

Arquitectura del sistema

El sistema sigue una arquitectura cliente/servidor desacoplada: el frontend Angular corre íntegramente en el navegador y se comunica por HTTPS con la API REST del backend, la cual persiste la información en una base de datos PostgreSQL gestionada de forma remota. Cada componente se despliega y escala de forma independiente.

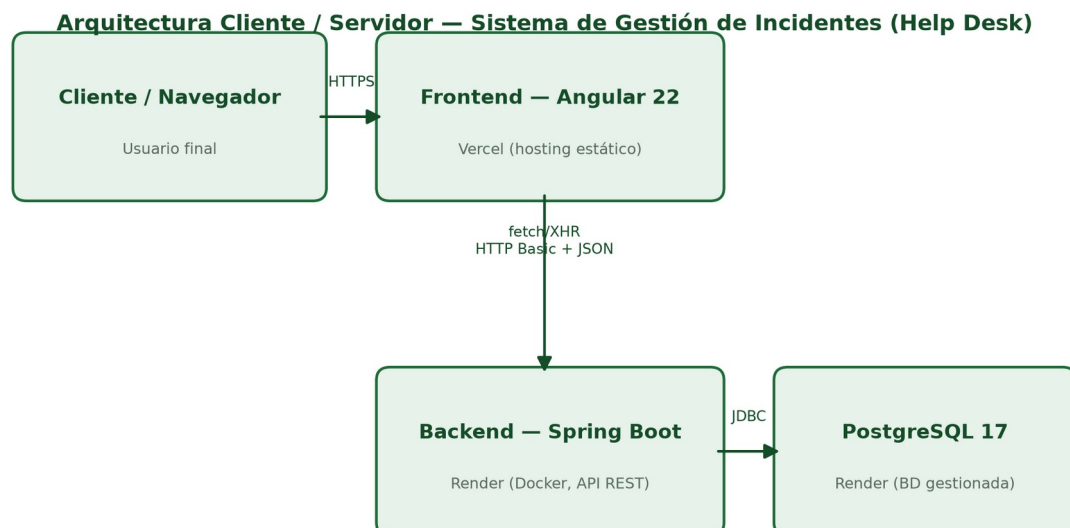


Figura 1. Diagrama de arquitectura cliente/servidor.



Componentes del frontend

Componente	Responsabilidad
Navegacion	Barra de navegación fija, menú responsivo y cierre de sesión.
Login	Captura las credenciales HTTP Basic y las guarda en sessionStorage.
Dashboard	KPIs (total, por estado), gráficos de barra por categoría/prioridad y últimos tickets.
RegistroIncidentes	Formulario reactivo con validaciones para crear tickets.
ListadoTickets	Tabla con filtros, edición de estado y eliminación (CRUD completo).
TicketService / Auth / authInterceptor / authGuard	Capa de servicios: consumo de la API, sesión y protección de rutas.

Integración y lógica de negocio

El servicio TicketService centraliza el consumo de los cinco endpoints REST mediante peticiones HTTP asíncronas (HttpClient + RxJS). Un interceptor adjunta la cabecera Authorization (HTTP Basic) únicamente a las peticiones dirigidas a la API configurada, y un guard de rutas impide el acceso a las vistas internas sin sesión iniciada.

Prevención de XSS

Angular escapa automáticamente cualquier valor mostrado mediante interpolación ({{ }}); el proyecto no utiliza [innerHTML] en ningún componente, por lo que el contenido ingresado por el usuario (título, descripción del incidente) nunca se interpreta como HTML o JavaScript. Adicionalmente, los campos de texto se recortan (trim) y se limitan en longitud (Validators.maxLength) antes de enviarse al backend, y las credenciales solo se almacenan en sessionStorage del navegador (nunca en localStorage ni en el código fuente).

Código principal del servicio de tickets (frontend)

```
import { Service, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../environments/environment';
import { Ticket, TicketRequest } from '../../models/ticket.model';

/**
 * Consume la API REST del backend (Actividad 8) mediante peticiones HTTP
 * asíncronas: los 5 endpoints CRUD sobre /tickets.
 */
@Service()
export class TicketService {
  private readonly http = inject(HttpClient);
  private readonly baseUrl = `${environment.apiUrl}/tickets`;

  listar(): Observable<Ticket[]> {
    return this.http.get<Ticket[]>(this.baseUrl);
  }

  buscar(id: number): Observable<Ticket> {
    return this.http.get<Ticket>(`${this.baseUrl}/${id}`);
  }

  crear(request: TicketRequest): Observable<Ticket> {
    return this.http.post<Ticket>(this.baseUrl, request);
  }

  actualizar(id: number, request: TicketRequest): Observable<Ticket> {
    return this.http.put<Ticket>(`${this.baseUrl}/${id}`, request);
  }

  eliminar(id: number): Observable<void> {
    return this.http.delete<void>(`${this.baseUrl}/${id}`);
  }
}
```



Despliegue en la nube

Componente	Servicio	URL pública
Base de datos (PostgreSQL)	Render	gestionada internamente
Backend (API REST)	Render (Docker)	https://helpdesk-datacenter-api.onrender.com
Frontend (SPA)	Vercel	https://helpdesk-datacenter-blush.vercel.app

El despliegue se automatizó con un Blueprint de Render (`render.yaml`), que aprovisiona la base de datos y el servicio web del backend a partir del Dockerfile del repositorio, y con Vercel para el sitio estático generado por el build de producción de Angular (`vercel.json`). Ambos servicios se conectan mediante variables de entorno: `CORS_ALLOWED_ORIGINS` en el backend habilita las peticiones desde el dominio del frontend, y `environment.prod.ts` en el frontend apunta a la URL pública del backend.



Sustentación en video

Enlace al video de sustentación: *PENDIENTE: enlace del video de sustentación (Drive/YouTube/Loom, sin restricción de acceso) — REQUISITO OBLIGATORIO, ver docs/GUION_VIDEO_SUSTENTACION.md*

El video muestra la arquitectura del sistema, el código fuente principal, el proceso de despliegue realizado en Render y Vercel, y una demostración en vivo del sistema funcionando desde la URL pública (creación, actualización y eliminación de tickets).



Capturas del sistema en producción

Capturas tomadas directamente desde las URLs públicas (no localhost), con un ticket real creado, listado y persistido en la base de datos remota.

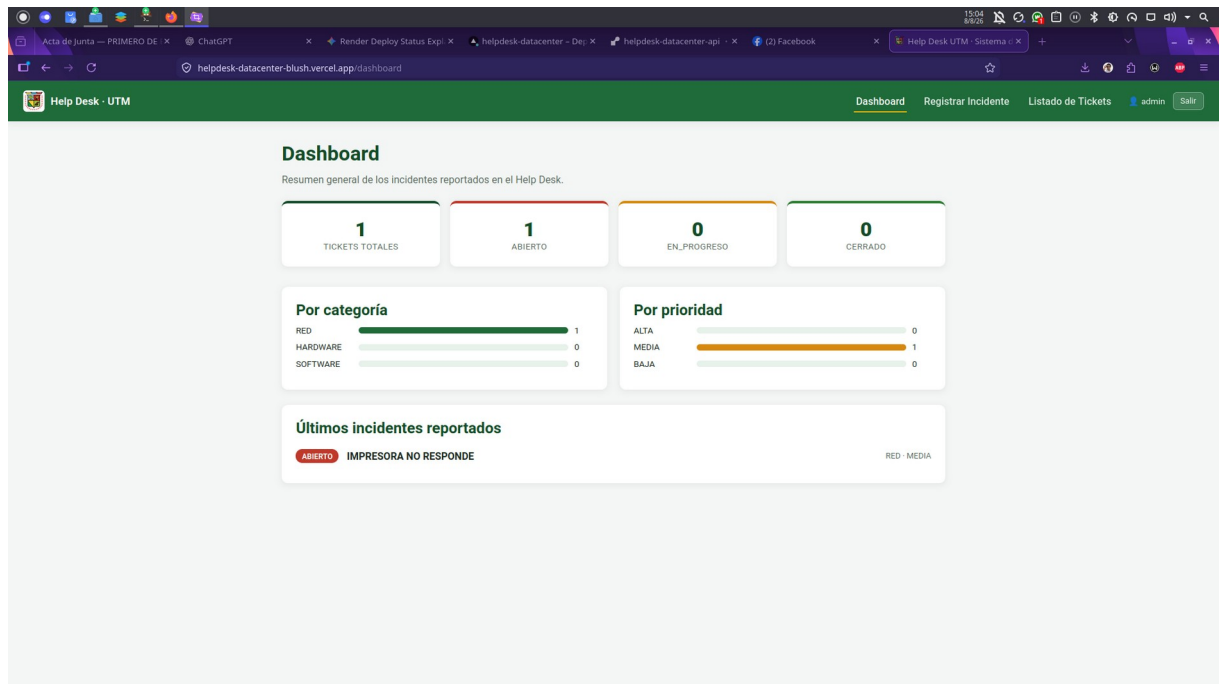


Figura 2. Dashboard en producción
(<https://helpdesk-datacenter-blush.vercel.app/dashboard>), con datos reales servidos por la API.



Listado de Tickets

Consulta, actualiza el estado o elimina los incidentes registrados.

Buscar por título... Todas las categorías Todos los estados Actualizar

#	TÍTULO	CATEGORÍA	PRIORIDAD	ESTADO	ACTUALIZADO	ACCIONES
1	IMPRESORA NO RESPONDE	RED	MEDIA	ABIERTO	8/8/26, 2:54 PM	 

Figura 3. Listado de Tickets en producción, mostrando el incidente creado en vivo.



Conclusiones técnicas

La integración full-stack evidenció la importancia de separar responsabilidades entre frontend y backend desplegados en dominios distintos: fue necesario habilitar CORS explícitamente en el backend para que el navegador permitiera las peticiones del frontend en producción. El uso de un Blueprint (render.yaml) y de vercel.json permitió reproducir el entorno de despliegue de forma declarativa y documentada, cumpliendo el requisito de que el sistema sea 100% funcional en la web sin depender de localhost.

Angular, mediante su escape automático de interpolaciones y el uso de formularios reactivos con validación, redujo significativamente el riesgo de vulnerabilidades XSS sin necesidad de librerías adicionales de sanitización.

Repositorio

Repositorio principal: <https://github.com/JBorys687/helpdesk-datacenter>

Rama de desarrollo integrada (unifica backend y frontend):

<https://github.com/JBorys687/helpdesk-datacenter/tree/develop>

Rama del frontend (Actividad 9):

<https://github.com/JBorys687/helpdesk-datacenter/tree/feature/frontend-app>

URL pública del sistema (frontend): <https://helpdesk-datacenter-blush.vercel.app>

URL pública de la API (backend): <https://helpdesk-datacenter-api.onrender.com>

Bibliografía

- Angular Team. (2026). Angular Documentation. <https://angular.dev/>
- Spring. (2026). Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/>
- Render. (2026). Render Documentation — Blueprints. <https://render.com/docs/blueprint-spec>
- Vercel. (2026). Vercel Documentation. <https://vercel.com/docs>
- OWASP Foundation. (2026). Cross Site Scripting (XSS) Prevention Cheat Sheet. <https://cheatsheetseries.owasp.org/>